Implementation Plan: Student Auth & Dashboard (React + Node.js + MySQL)

This plan outlines the architecture, data models, API contracts, frontend views, and deployment strategy to build a Student Registration, Login, and Dashboard application.

1. System Architecture

2. Directory Structure

We will structure the application cleanly with a separated `client` and `server`:

```
text
simple signup and login form and dashboard/
├── client/                      # React frontend (Vite)
│   ├── src/
│   │   ├── components/
│   │   │   └── ProtectedRoute.jsx
│   │   ├── pages/
│   │   │   ├── Register.jsx
│   │   │   ├── Login.jsx
│   │   │   └── Dashboard.jsx
│   │   ├── services/
│   │   │   └── api.js          # Axios / Fetch client configured with JWT
│   │   ├── App.jsx             # React router setup
│   │   ├── main.jsx
│   │   └── index.css           # Modern, responsive styles
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
│
├── server/                      # Node.js + Express backend
│   ├── src/
│   │   ├── config/
│   │   │   └── db.js           # MySQL connection pool (mysql2/promise)
│   │   ├── controllers/
│   │   │   ├── authController.js
│   │   │   └── studentController.js
│   │   ├── middlewares/
│   │   │   └── authMiddleware.js # JWT verification
│   │   ├── routes/
│   │   │   ├── authRoutes.js
│   │   │   └── studentRoutes.js
│   │   ├── schema.sql          # SQL schema definition & seed data
│   │   └── server.js           # App entrypoint & CORS config
│   ├── .env.example
│   ├── package.json
│   └── README.md
│
└── README.md
```

3. Database Schema (MySQL)

Table name: `students`

Field	Type	Modifiers	Description
`id`	`INT`	`AUTO_INCREMENT PRIMARY KEY`	Unique student ID
`name`	`VARCHAR(100)`	`NOT NULL`	Full name of the student
`email`	`VARCHAR(150)`	`NOT NULL UNIQUE`	Login email address
`password`	`VARCHAR(255)`	`NOT NULL`	Hashed password (via `bcryptjs`)
`roll_number`	`VARCHAR(50)`	`NOT NULL UNIQUE`	Academic roll / registration number
`course`	`VARCHAR(100)`	`NOT NULL`	Degree or program (e.g., B.Tech CS)
`department`	`VARCHAR(100)`	`NOT NULL`	Department / Faculty
`phone`	`VARCHAR(20)`	`NULL`	Contact phone number
`created_at`	`TIMESTAMP`	`DEFAULT CURRENT_TIMESTAMP`	Account creation timestamp

4. Backend Implementation Plan

Dependencies

- `express`: REST API web framework
- `mysql2`: MySQL driver with promise support and connection pooling
- `bcryptjs`: Secure one-way password hashing (salt rounds: 10)
- `jsonwebtoken`: Stateless token generation for session authorization
- `cors`: Cross-Origin Resource Sharing handling
- `dotenv`: Environment variable management

API Endpoints

1. `POST /api/auth/register`
    - **Request Body**: `{ name, email, password, roll_number, course, department, phone }`
    - **Validation**: Check required fields, valid email format, password minimum length.

- **Execution**: Check if email or roll number already exists. Hash password. Insert record into `students`.
- **Response**: `201 Created` with success message.

2. `POST /api/auth/login`
   - **Request Body**: `{ email, password }`
   - **Execution**: Retrieve student by email. Compare password hash with `bcrypt.compare`.
   - **Response**: `200 OK` with `{ token, student: { id, name, email, roll_number } }`.
3. `GET /api/student/profile`
   - **Headers**: `Authorization: Bearer <jwt_token>`
   - **Middleware**: Decodes JWT, attaches `req.user = { id, email }`.
   - **Execution**: Queries fresh student record from database (excluding password): `SELECT id, name, email, roll_number, course, department, phone, created_at FROM students WHERE id = ?`.
   - **Response**: `200 OK` with full student details.

5. Frontend Implementation Plan (React)

Features & Pages

1. **Registration View (`/register`)**:
   - Modern styled card with form inputs: Name, Email, Password, Roll Number, Course, Department, Phone.
   - Client-side validation with instant inline error feedback.
   - Success redirect to `/login` upon completion.
2. **Login View (`/login`)**:
   - Clean credentials form: Email and Password.
   - Handles authentication errors (e.g. invalid credentials).
   - On success, stores JWT in `localStorage` and redirects to `/dashboard`.
3. **Protected Student Dashboard (`/dashboard`)**:
   - Guarded by `<ProtectedRoute>`: redirects unauthenticated visitors to `/login`.
   - Fetches live student profile using the stored JWT token.
   - Displays student identification badge / profile card with:

        - Avatar placeholder / Initials
        - Name, Roll Number, Email, Course, Department, Phone, Joined Date.
    - **Logout Action**: Clears token from `localStorage` and navigates back to login.

6. Deployment Plan on Render

Render does not offer a native managed MySQL service on its free tier (Render provides managed PostgreSQL), but Render connects seamlessly to free cloud MySQL providers (such as **Aiven for MySQL**, **PlanetScale**, or **Clever Cloud**).

Deployment Architecture

- **Option A (Separated Deploy - Standard)**:
    - **Database**: Free cloud MySQL instance on Aiven or Clever Cloud.
    - **Backend**: Render Web Service (Node.js runtime).
    - **Frontend**: Render Static Site (built from Vite).
- **Option B (Unified Single Service - Simplest)**:
    - Backend serves the compiled React static files from `client/dist`.
    - Only requires a single Render Web Service + External MySQL connection.

Step-by-Step Render Deployment Guide

Step 1: Set Up Cloud MySQL Database

1. Create a free account at Aiven.io or Clever Cloud.
2. Create a free MySQL database instance (e.g., `student_db`).
3. Note your connection details:
    - Host
    - Port
    - User
    - Password
    - Database name
4. Run the `schema.sql` script on the database (via MySQL Workbench, phpMyAdmin, or CLI) to create the `students` table.

Step 2: Push Repository to GitHub

1. Initialize git in the root folder:

```
bash
git init
git add .
git commit -m "feat: complete student auth and dashboard"
```

2. Create a GitHub repository and push your code.

Step 3: Deploy Backend on Render

1. Go to Render Dashboard and click **New +** -> **Web Service**.
2. Connect your GitHub repository.
3. Configure the Web Service:
   - **Root Directory**: `server`
   - **Runtime**: `Node`
   - **Build Command**: `npm install`
   - **Start Command**: `npm start`
4. Add **Environment Variables** in Render settings:
   - `PORT`: `5000` (or leave default Render port)
   - `DB_HOST`: `<Your Cloud MySQL Host>`
   - `DB_PORT`: `<Your Cloud MySQL Port>`
   - `DB_USER`: `<Your Cloud MySQL User>`
   - `DB_PASSWORD`: `<Your Cloud MySQL Password>`
   - `DB_NAME`: `<Your Cloud MySQL Database Name>`
   - `JWT_SECRET`: `<A strong random secret string>`
   - `CLIENT_URL`: `<Your Render Frontend URL once deployed>`
5. Click **Create Web Service**. Once deployed, copy the backend URL (e.g. `https://student-api.onrender.com`).

Step 4: Deploy Frontend on Render

1. In Render Dashboard, click **New +** -> **Static Site**.
2. Connect the same GitHub repository.
3. Configure the Static Site:

- **Root Directory**: `client`
- **Build Command**: `npm install && npm run build`
- **Publish Directory**: `dist`

4. Add **Environment Variable**:
   - `VITE_API_URL` : `https://student-api.onrender.com` (Your backend Render URL)
5. Set up **Redirects/Rewrites** (for Single Page App routing):
   - Under the **Redirects/Rewrites** tab, add a rule:
     - **Source**: `/*`
     - **Destination**: `/index.html`
     - **Action**: `Rewrite`
6. Click **Create Static Site**.

7. Execution Order

1. **Phase 1: Backend Setup & Database Layer**
   - Initialize Node project in `server/` .
   - Setup MySQL connection pool, `schema.sql` , and test connection script.
   - Implement JWT authentication middleware.
   - Implement register, login, and student profile endpoints.
2. **Phase 2: Frontend Setup & UI**
   - Initialize React app in `client/` using Vite.
   - Setup React Router with protected routes.
   - Create Register, Login, and Student Profile Dashboard pages with modern, responsive CSS.
3. **Phase 3: Integration & Local Verification**
   - Test full flow locally: registration -> login -> fetch student details on dashboard -> logout.
4. **Phase 4: Render Readiness**
   - Add `.env.example` , build scripts, and production configurations.